

Chunking Strategies

& Vector Databases

A Complete Beginner-to-Advanced Guide with Comparisons, Code & Real-World Examples

8 CHUNK TYPES

7 VECTOR DBs

SIDE-BY-SIDE COMPARE

CODE EXAMPLES

Table of Contents

1	Why Chunking Matters The foundation — what chunking is and why bad chunking kills RAG quality
2	Fixed-Size Chunking The simplest approach — character or token count based splitting
3	Recursive Character Chunking LangChain's default — hierarchy-aware splitting with fallback
4	Sentence-Based Chunking Split at natural sentence boundaries for cleaner chunks
5	Semantic Chunking AI-powered splitting — detects topic changes to create coherent chunks
6	Markdown & HTML Chunking Structure-aware splitting for documents with headers
7	Agentic / Proposition Chunking Advanced — extract atomic facts instead of raw text chunks
8	Parent-Child Chunking Two-level indexing — retrieve small, send large to LLM
9	Chunking Comparison Tables Side-by-side: quality, speed, complexity, when to use each

10

What is a Vector Database?

Complete theory — from vectors to ANN search algorithms

11

7 Vector Databases Compared

FAISS vs Chroma vs Qdrant vs Pinecone vs Weaviate vs pgvector vs Redis

12

Choosing the Right Combination

Decision guide + recommended stacks for every use case

Chapter 1

Why Chunking Matters

The foundation of every RAG system

1.1 What is Chunking?

Chunking is the process of breaking a large document into smaller, manageable pieces before storing them in a vector database. Each piece is called a 'chunk'. These chunks are what get embedded (converted to vectors) and later retrieved when a user asks a question.

■ *Analogy: Imagine you have a 500-page encyclopaedia and someone asks: 'What is the boiling point of water?' You would not photocopy all 500 pages and hand them over. You would find the right page, rip it out, and hand just that page. Chunking is how you pre-prepare all possible 'pages to rip out'.*

1.2 Why Not Just Use the Whole Document?

There are two hard reasons and one soft reason why you cannot skip chunking:

- **Hard reason 1 — Embedding model limit:** Embedding models like all-MiniLM-L6-v2 can only process up to 256 tokens (~190 words) at once. A 50-page PDF is ~25,000 tokens. You physically cannot embed the whole document as one unit.
- **Hard reason 2 — Retrieval precision:** If your entire document is one vector, every question retrieves the same thing — the whole document. You lose all ability to find the specific section that answers the question.
- **Soft reason — LLM context cost:** Even if your LLM supports 200,000 tokens, stuffing an entire document into every prompt is slow and expensive. Retrieving just the 3-4 relevant chunks is much faster and cheaper.

1.3 What Makes a Good Chunk?

Property	What It Means	Bad Example	Good Example
Self-contained	The chunk makes sense on its own, without needing surrounding chunks	'...as mentioned above, the policy requires...' (what policy? what was mentioned above?)	'The refund policy requires customers to return items within 30 days of purchase.'
Focused	The chunk covers one topic, not three	A 2,000-char chunk covering pricing, delivery AND refunds all mixed together	One 400-char chunk covering only the refund policy

Property	What It Means	Bad Example	Good Example
Correct size	Not too small (no context) and not too large (diluted meaning)	'30 days.' (5 chars — meaningless alone)	'Customers may return items within 30 days. Items must be unused and in original packaging.' (200 chars)
Overlapping boundaries	Sentences that span two chunks still appear in both	Key sentence cut in half across chunk boundary — half the meaning is in each chunk	50-100 char overlap ensures boundary sentences appear in full in at least one chunk

1.4 Chunking Strategy Impact on Quality

Chunking is arguably the single biggest lever for RAG quality improvement — more impactful than the choice of LLM in many cases. Here is a concrete illustration:

■ Example: Document: A 20-page employee handbook. Question: 'How many sick days am I allowed?'
Bad chunking (500 chars, no overlap): The answer 'Employees receive 12 sick days per year' might be split across two chunks. Neither chunk contains the complete answer. The LLM retrieves incomplete context and either answers incorrectly or says 'not found'. Good chunking (semantic): The entire sick leave section stays together as one chunk. The complete policy is retrieved. The LLM answers correctly.

Chapter 2

Fixed-Size Chunking

The simplest approach — split by character or token count

2.1 What Is It?

Fixed-size chunking is the most basic strategy. You choose a number (e.g., 500 characters) and cut the document every time you reach that count. Each chunk is exactly that size (the last chunk may be smaller). You can optionally add overlap — a number of characters shared between consecutive chunks.

■ *Analogy: Cutting a long rope into pieces every 50cm. Simple, predictable, but does not care about knots (natural boundaries) in the rope.*

2.2 Character-Based Fixed Chunking

Character-Based Fixed Chunking

```
from langchain_text_splitters import CharacterTextSplitter

# Basic character-based splitting
splitter = CharacterTextSplitter(
    separator="\n",      # where to try to split first
    chunk_size=500,      # max characters per chunk
    chunk_overlap=50     # characters shared between adjacent chunks
)

text = """The refund policy allows customers to return items within 30 days.
Items must be in original packaging. Digital goods are non-refundable.
For hardware, please contact support within 7 days of delivery.
Extended warranty covers defects for up to 2 years from purchase date."""

chunks = splitter.split_text(text)
for i, chunk in enumerate(chunks):
    print(f"Chunk {i+1} ({len(chunk)} chars): {chunk[:60]}...")

# Output (approximate):
# Chunk 1 (89 chars): The refund policy allows customers to return items within 30 ...
# Chunk 2 (91 chars): Items must be in original packaging. Digital goods are non-r...
# Chunk 3 (82 chars): For hardware, please contact support within 7 days of delive...
```

2.3 Token-Based Fixed Chunking

Instead of counting characters, you count tokens (the units LLMs actually use). This is more accurate because 500 characters can be anywhere from 80 to 200 tokens depending on the words used. Token-based chunking ensures you never exceed the LLM's context limits.

Token-Based Fixed Chunking

```
from langchain_text_splitters import TokenTextSplitter

# Token-based splitting (uses tiktoken under the hood)
splitter = TokenTextSplitter(
    chunk_size=150,      # 150 tokens ≈ ~115 words ≈ ~600 characters
    chunk_overlap=20     # 20 token overlap between chunks
)

chunks = splitter.split_text(text)

# Why token-based is better than character-based:
# "cryptocurrency"    = 1 token  (1 word, 14 chars)
# "I am happy"        = 3 tokens  (3 words, 10 chars)
# Chinese/Arabic text: 1 character can = 1 token
# So character count is unreliable for estimating LLM context usage

# Rule of thumb: 1 token ≈ 4 English characters ≈ 0.75 words
# 500 chars ≈ 125 tokens   |   1000 chars ≈ 250 tokens
```

2.4 Chunk Overlap — Why It Matters

Overlap Explained

```
# WITHOUT overlap (chunk_overlap=0):
# Chunk 1: "The customer must submit a refund request within"
# Chunk 2: "30 days of receiving the item to qualify for reimbursement."
#
# The key information is SPLIT across two chunks.
# If retrieval returns Chunk 1: LLM sees no timeframe.
# If retrieval returns Chunk 2: LLM sees no context about what the 30 days refers to.

# WITH overlap (chunk_overlap=50):
# Chunk 1: "The customer must submit a refund request within"
# Chunk 2: "request within 30 days of receiving the item to qualify for reimbursement."
#           ^^^^^^^^^^^^^^^^^^ — the overlap ensures Chunk 2 contains the full sentence!

# Recommended overlap: 10-20% of chunk_size
# chunk_size=500   → overlap=50-100
# chunk_size=1000  → overlap=100-200
# chunk_size=2000  → overlap=200-400
```

2.5 Pros, Cons & When to Use

Aspect	Details
■ Pros	Simplest to implement. Very fast — no AI needed. Predictable chunk sizes. Works for any text format.
■ Cons	Ignores all natural language boundaries. Can cut sentences in half. Can split a topic across chunks. Poor retrieval quality for nuanced questions.
Best used when	You just need something working fast. Very large documents where speed matters. Documents with uniform structure (e.g., logs, CSV-like text).

Aspect	Details
Avoid when	Document has clear structure (headers, paragraphs). Answer quality is critical. Documents are short enough for better strategies.
Recommended settings	chunk_size: 500-1000 chars (or 150-300 tokens). overlap: 10-15% of chunk_size.

Chapter 3

Recursive Character Chunking

LangChain's default — smarter boundary detection

3.1 What Is It?

Recursive Character Text Splitting is an improvement over fixed-size chunking. Instead of always cutting at a fixed character count, it tries a list of separators in order of preference — from 'best' (paragraph break) to 'last resort' (single character). It only falls back to a less ideal separator if the text cannot be split cleanly at the preferred one.

■ *Analogy: Instead of cutting a rope every 50cm regardless of knots, you look for natural knots (paragraph breaks) first. If a section between knots is too long, you look for smaller knots (line breaks) within it. Only if everything fails do you cut mid-strand.*

3.2 The Separator Hierarchy

Separator Hierarchy Logic

```
# Default separator order – tried in this exact sequence:
separators = [
    "\n\n",    # 1st choice: paragraph break (double newline)
               # "...end of paragraph.\n\nStart of new paragraph..."
    "\n",      # 2nd choice: single line break
    ".",       # 3rd choice: sentence end
    ",",       # 4th choice: clause boundary
    " ",       # 5th choice: word boundary
    "",        # Last resort: split at any character
]

# How it works step by step:
# Given: chunk_size=200, a text block of 800 characters

# Step 1: Try splitting on \n\n
# Result: pieces of [150, 250, 400] characters
# The 250 and 400 are > 200 → need further splitting

# Step 2: Take the 250-char piece. Try \n
# Result: [120, 130] → both <= 200 → DONE for this piece

# Step 3: Take the 400-char piece. Try \n
# Result: [180, 220] → 220 still > 200 → continue

# Step 4: Take the 220-char piece. Try "."
# Result: [100, 120] → both <= 200 → DONE

# Final chunks: [150, 120, 130, 180, 100, 120] – all ≤ 200 chars
```

3.3 Code Example

RecursiveCharacterTextSplitter

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100,
    separators=["\n\n", "\n", ".", ",", " ", ""], # explicit list
    length_function=len, # count characters (default)
    is_separator_regex=False, # treat separators as plain strings
)

# Split a list of Document objects (from a PDF loader):
chunks = splitter.split_documents(documents)

# Split raw text:
chunks = splitter.split_text(my_text)

# ■■ Tiktoken-aware version (counts tokens, not characters) ■■
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    model_name="gpt-4", # use GPT-4's tokenizer
    chunk_size=400, # 400 tokens ≈ 300 words
    chunk_overlap=80,
)

# This ensures chunks never exceed 400 GPT-4 tokens
# regardless of how many characters they contain
```

3.4 Pros, Cons & When to Use

Aspect	Details
■ Pros	Respects natural language boundaries. No AI needed — pure string logic. Fast. Good default for most use cases. Configurable separator list.
■ Cons	Still does not understand meaning. Can split a paragraph about topic A and one about topic B into the same chunk. Separator list needs tuning for non-English or structured text.
Best used when	General-purpose RAG. Prototyping. Documents with clear paragraph/line structure. Most real-world use cases — this is the right default.
Avoid when	Documents have no consistent formatting. You need semantically coherent chunks (use SemanticChunker instead).
Recommended settings	chunk_size: 1000 chars or 300-400 tokens. chunk_overlap: 15-20% of chunk_size.

Chapter 4

Sentence-Based Chunking

Split at sentence boundaries — cleaner, more natural chunks

4.1 What Is It?

Sentence-based chunking uses a proper sentence tokenizer (from NLP libraries like NLTK or spaCy) to detect where sentences end and begin. It then groups a certain number of sentences together into each chunk. This is better than character splitting because it never cuts a sentence in half.

■ *Analogy: Instead of cutting every 50cm, you cut only between complete sentences. No sentence is ever split across two chunks.*

4.2 NLTK Sentence Splitter

NLTK Sentence Splitter

```
# pip install nltk
import nltk
nltk.download("punkt", quiet=True)    # download sentence tokenizer model

from langchain_text_splitters import NLTKTextSplitter

splitter = NLTKTextSplitter(
    chunk_size=500,    # target chunk size in characters
)

text = """The company was founded in 1998 by two engineers.
It grew rapidly through the 2000s. By 2010, revenue exceeded $1 billion.
The headquarters is located in San Francisco, California.
The company employs over 50,000 people worldwide."""

chunks = splitter.split_text(text)
# Each chunk will contain complete sentences only.
# No sentence will be split across chunk boundaries.
```

4.3 spaCy Sentence Splitter

spaCy Sentence Splitter

```
# pip install spacy
# python -m spacy download en_core_web_sm

from langchain_text_splitters import SpacyTextSplitter

splitter = SpacyTextSplitter(
```

```
chunk_size=500,
pipeline="en_core_web_sm",    # spaCy language model
)

chunks = splitter.split_text(text)

# spaCy advantages over NLTK:
# - Better at handling abbreviations (U.S.A., Dr., etc.)
# - Better multilingual support
# - More accurate on complex sentences
# - Understands named entities (useful for metadata)

# spaCy disadvantages:
# - Requires downloading a language model (~50MB)
# - Slower than NLTK
# - Overkill for simple English text
```

4.4 Sentence Window Chunking (Advanced Variant)

A popular variant: embed individual sentences for precise retrieval, but when a sentence is retrieved, expand it to include surrounding sentences (the 'window') before passing to the LLM. This gives precise retrieval but rich context for generation.

Sentence Window Concept

```
# Concept: Sentence Window Retrieval
# Step 1: Split into individual sentences and embed each one
sentences = ["Sentence 1.", "Sentence 2.", "Sentence 3.", "Sentence 4.", "Sentence 5."]

# Step 2: At query time, retrieve the MOST RELEVANT sentence
# e.g., "Sentence 3." is retrieved as most similar to query

# Step 3: Expand to a window of ±2 sentences around it
window = ["Sentence 1.", "Sentence 2.", "Sentence 3.", "Sentence 4.", "Sentence 5."]
#          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
#          This is what gets sent to the LLM as context

# Why this is powerful:
# - Retrieval is PRECISE (individual sentences)
# - LLM context is RICH (full surrounding paragraph)
# This is implemented in LlamaIndex as SentenceWindowNodeParser
```

4.5 Pros, Cons & When to Use

Aspect	Details
■ Pros	Never splits a sentence. More natural chunk boundaries. Slightly better retrieval than character splitting. Sentence window variant gives best of both worlds.
■ Cons	Requires NLTK or spaCy installation. Slower than pure string splitting. Still does not understand semantic topic boundaries. Short sentences create tiny chunks.
Best used when	Documents have clear prose structure (articles, reports, manuals). You want cleaner chunks than RecursiveCharacterTextSplitter. Multilingual documents (spaCy).

Aspect	Details
Avoid when	Documents are code, logs, or structured data. Speed is the priority. Short sentences would create many tiny chunks.
Recommended settings	chunk_size: 500-800 chars. window_size: 3-5 sentences on each side for sentence window variant.

Chapter 5

Semantic Chunking

AI-powered — detects topic changes to create coherent chunks

5.1 What Is It?

Semantic chunking is the most intelligent chunking strategy. Instead of splitting on character counts or punctuation, it uses an embedding model to detect where the topic changes in a document. When two consecutive sentences have very different embedding vectors (meaning they are about different topics), that is a split point.

■ *Analogy: Imagine reading a book and marking a new section every time the subject changes — even if there is no heading. You read sentence by sentence, and when you feel the topic has shifted, you draw a dividing line. Semantic chunking does this automatically using AI.*

5.2 How It Works Internally

SemanticChunker Internals

```
# Internal process of SemanticChunker:

# Step 1: Split text into individual sentences
sentences = [
    "The refund policy allows 30-day returns.",      # about returns
    "Items must be in original packaging.",          # about returns
    "Digital downloads cannot be refunded.",         # about returns
    "Our delivery service covers 50 countries.",     # << TOPIC CHANGE
    "Standard shipping takes 3-5 business days.",    # about shipping
    "Express delivery arrives within 24 hours.",     # about shipping
]

# Step 2: Embed each sentence into a vector
embeddings = [embed(s) for s in sentences]

# Step 3: Calculate similarity between CONSECUTIVE sentences
# similarity(sentence 1, sentence 2) = 0.89 # very similar (both about returns)
# similarity(sentence 2, sentence 3) = 0.82 # similar (still returns)
# similarity(sentence 3, sentence 4) = 0.21 # << BIG DROP = topic change!
# similarity(sentence 4, sentence 5) = 0.91 # very similar (both about shipping)
# similarity(sentence 5, sentence 6) = 0.88 # similar (still shipping)

# Step 4: Find split points where similarity drops below threshold
# Split detected between sentence 3 and sentence 4

# Step 5: Group sentences into chunks at split points
# Chunk 1: sentences 1-3 (refund policy)
# Chunk 2: sentences 4-6 (delivery info)
```

5.3 LangChain SemanticChunker Code

SemanticChunker Usage

[illegible]

5.4 Pros, Cons & When to Use

Aspect	Details
■ Pros	Best retrieval quality of all chunking strategies. Chunks are semantically coherent — each covers exactly one topic. No manual tuning of chunk_size needed. Handles documents with irregular structure.
■ Cons	Slowest indexing — requires embedding every sentence. Requires an embedding model at chunking time. Can produce very variable chunk sizes. Harder to debug than simple strategies.

Aspect	Details
Best used when	Answer quality is the top priority. Documents cover many different topics in irregular order. You are building a production system and can afford slower indexing. Legal, medical, or technical documents where precision is critical.
Avoid when	You need fast indexing. Documents are already well-structured with clear headers. You are prototyping and need something working in 5 minutes.
Recommended settings	<code>breakpoint_threshold_type</code> : 'percentile' with <code>amount=95</code> is the safest starting point. Always save the index to disk after building.

Chapter 6

Markdown & HTML Chunking

Structure-aware splitting — use document headings as natural boundaries

6.1 What Is It?

Markdown and HTML documents have explicit structure: headings (# H1, ## H2), sections, tables, and code blocks. Structure-aware splitters use these markers as split points, creating chunks that correspond to actual document sections. The heading hierarchy is preserved as metadata on each chunk.

■ Example: A technical documentation page with sections '## Installation', '## Configuration', '## API Reference' — each section becomes its own chunk, with the heading stored as metadata. Retrieval then finds the exact section instead of an arbitrary character-cut piece.

6.2 MarkdownHeaderTextSplitter

MarkdownHeaderTextSplitter

```
from langchain_text_splitters import MarkdownHeaderTextSplitter

# Define which headers to split on and their metadata key names
headers_to_split_on = [
    ("#", "h1"),    # H1 headings → stored as metadata["h1"]
    ("##", "h2"),   # H2 headings → stored as metadata["h2"]
    ("###", "h3"),  # H3 headings → stored as metadata["h3"]
]

splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=headers_to_split_on,
    strip_headers=False,  # keep header text in the chunk content
)

markdown_text = """
# Company Handbook

## Refund Policy
Customers may return items within 30 days of purchase.
Items must be unused and in original packaging.

## Shipping Policy
We offer standard and express delivery.
Standard shipping takes 3-5 business days.
"""

chunks = splitter.split_text(markdown_text)

# Results:
# Chunk 1:
```

```
# content: "Customers may return items within 30 days..."
# metadata: {"h1": "Company Handbook", "h2": "Refund Policy"}
#
# Chunk 2:
# content: "We offer standard and express delivery..."
# metadata: {"h1": "Company Handbook", "h2": "Shipping Policy"}

# The metadata is GOLD for retrieval filtering:
# vectorstore.similarity_search(q, filter={"h2": "Refund Policy"})
```

6.3 HTMLHeaderTextSplitter (for Web Content)

HTMLHeaderTextSplitter

```
from langchain_text_splitters import HTMLHeaderTextSplitter

headers_to_split_on = [
    ("h1", "Header 1"),
    ("h2", "Header 2"),
    ("h3", "Header 3"),
]

splitter = HTMLHeaderTextSplitter(headers_to_split_on=headers_to_split_on)

# For a URL:
html_string = """<html><body>
<h1>Product Manual</h1>
<h2>Getting Started</h2>
<p>Connect the device to power using the included cable.</p>
<h2>Troubleshooting</h2>
<p>If the device does not turn on, check the power connection.</p>
</body></html>"""

chunks = splitter.split_text(html_string)
# Each <h2> section becomes its own chunk with header as metadata
```

6.4 Combining Structural + Recursive Splitting

A powerful pattern: use `MarkdownHeaderTextSplitter` to split at headers first, then apply `RecursiveCharacterTextSplitter` to any chunks that are still too large. This gives you structure-awareness plus size control.

Two-Stage Splitting

```
from langchain_text_splitters import (
    MarkdownHeaderTextSplitter,
    RecursiveCharacterTextSplitter
)

# Stage 1: Split by headers
md_splitter = MarkdownHeaderTextSplitter(headers_to_split_on=[
    ("#", "h1"), ("##", "h2"), ("###", "h3")
])
header_chunks = md_splitter.split_text(markdown_text)

# Stage 2: Further split any large chunks
```

```
char_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=100
)
final_chunks = char_splitter.split_documents(header_chunks)
# Metadata from headers is preserved on all resulting chunks!
```

6.5 Pros, Cons & When to Use

Aspect	Details
■ Pros	Chunks map to real document sections. Heading metadata enables precise retrieval filtering. No AI model needed. Very fast. Great for documentation, wikis, READMEs.
■ Cons	Only works for well-structured Markdown/HTML. PDF documents rarely have proper heading structure after text extraction. Sections without headers are handled as one big block.
Best used when	Technical documentation, wikis, Confluence pages, knowledge bases, README files, any Markdown or HTML content with proper heading structure.
Avoid when	Scanned PDFs, plain-text documents without structure, documents where headings are inconsistent.

Chapter 7

Agentic / Proposition Chunking

Advanced — use an LLM to extract atomic facts as chunks

7.1 What Is It?

Proposition chunking is an advanced technique where you use an LLM to read each passage and extract individual atomic facts (called propositions) from it. Each proposition is a single, standalone factual statement. These propositions become your chunks — they are self-contained, precise, and perfect for embedding.

■ *Analogy: Instead of cutting a news article into paragraphs, you have an assistant read it and write out every single fact as a separate flashcard. Each flashcard says one complete thing and makes sense on its own.*

7.2 How It Works

Proposition Extraction

```
# INPUT (a paragraph):
passage = """
Apple Inc. was founded in 1976 by Steve Jobs, Steve Wozniak, and Ronald Wayne.
The company is headquartered in Cupertino, California. Apple's first product
was the Apple I personal computer. As of 2024, Apple is the most valuable
company in the world with a market cap exceeding $3 trillion.
"""

# Proposition Extraction Prompt sent to an LLM:
prompt = """
Decompose the following passage into a list of atomic, self-contained facts.
Each fact should:
- Be a single complete sentence
- Make sense without reading other facts
- Not use pronouns like 'it', 'they', 'the company' — use proper nouns

Passage: {passage}

Return as a JSON list of strings.
"""

# OUTPUT (what the LLM returns):
propositions = [
    "Apple Inc. was founded in 1976.",
    "Apple Inc. was founded by Steve Jobs, Steve Wozniak, and Ronald Wayne.",
    "Apple Inc. is headquartered in Cupertino, California.",
    "Apple's first product was the Apple I personal computer.",
    "As of 2024, Apple Inc. is the most valuable company in the world.",
```

```
    "Apple Inc.'s market capitalization exceeded $3 trillion as of 2024.",
]

# Each of these becomes an embedded chunk in your vector store!
# MUCH more precise than character-based chunks
```

7.3 LangChain Implementation

Proposition Chunking Code

```
from langchain_core.output_parsers import JsonOutputParser
from langchain_core.prompts import PromptTemplate
from langchain_community.llms import Ollama
from langchain_core.documents import Document

llm = Ollama(model="mistral") # use a capable model

PROP_PROMPT = PromptTemplate.from_template("""
Extract all atomic facts from the text below.
Each fact must be a complete, standalone sentence with no pronouns.
Return ONLY a JSON array of strings. No explanation.

Text: {text}
""")

def extract_propositions(text: str) -> list[Document]:
    chain = PROP_PROMPT | llm | JsonOutputParser()
    props = chain.invoke({"text": text})
    # Wrap each proposition in a Document
    return [Document(page_content=p, metadata={"source": "propositions"})
            for p in props]

# Process all your document chunks:
all_propositions = []
for doc in raw_chunks:
    props = extract_propositions(doc.page_content)
    all_propositions.extend(props)

# Build vector store from propositions:
vectorstore = FAISS.from_documents(all_propositions, embeddings)

# ■ WARNING: This calls the LLM once per chunk.
# For 500 chunks -> 500 LLM calls. Can take 10-30 minutes.
# Cost: if using GPT-4o-mini, ~$0.01-0.05 for a 100-page document.
```

7.4 Pros, Cons & When to Use

Aspect	Details
■ Pros	Highest possible retrieval precision. Every chunk is self-contained and unambiguous. Eliminates context-dependency between chunks. Dramatically reduces hallucination.
■ Cons	Very slow indexing — one LLM call per passage. Can be expensive if using a paid LLM API. Requires a capable LLM (weak models extract incorrect propositions). Chunks may lose writing style/nuance.

Aspect	Details
Best used when	Legal documents, contracts, medical records — anywhere where precision is non-negotiable. Knowledge bases where accuracy > speed. Long-term production systems where indexing happens offline once.
Avoid when	You need fast indexing. Budget is limited. Documents are creative writing or narrative where factual extraction loses important context.

Chapter 8

Parent-Child Chunking

Two-level indexing — precise retrieval, rich context for generation

8.1 The Core Idea

Parent-Child chunking (also called Small-to-Large or Multi-Level chunking) creates two sets of chunks from the same document:

- **Child chunks (small):** ~100-200 character pieces. Very focused. These are embedded and stored in the vector database. Small chunks give precise retrieval — when a user asks a specific question, the small chunk that exactly answers it is returned.
- **Parent chunks (large):** ~500-2000 character pieces. These are NOT embedded. They are stored in a separate store indexed by ID. When a child chunk is retrieved, its parent chunk ID is used to look up the full parent — which is what gets sent to the LLM.

■ *Analogy: A library catalogue shows you individual book pages (child chunks) that match your search. But when you go to read, you get the full chapter (parent chunk) that page belongs to — not just that one page in isolation.*

8.2 Why This Solves a Real Problem

There is a fundamental tension in RAG chunking:

- **For retrieval:** Small chunks are better. A 50-word chunk about refunds will match a refund question much more precisely than a 500-word chunk about general policies.
- **For generation:** Large chunks are better. The LLM needs surrounding context to give a complete, coherent answer. Just the one sentence that answers the question is often not enough.

Parent-Child chunking solves both sides: use small chunks for retrieval, but return the large parent chunk to the LLM. Best of both worlds.

8.3 LangChain Implementation

ParentDocumentRetriever

```
from langchain.storage import InMemoryStore
from langchain.retrievers import ParentDocumentRetriever
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
```

```
# Child splitter – creates SMALL chunks for embedding
child_splitter = RecursiveCharacterTextSplitter(
    chunk_size=200,      # small – for precise retrieval
    chunk_overlap=20
)

# Parent splitter – creates LARGE chunks sent to LLM
parent_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,     # large – rich context for the LLM
    chunk_overlap=100
)

# Vector store for child chunks (embedded and searched)
vectorstore = FAISS.from_documents([], embeddings)

# Document store for parent chunks (looked up by ID)
doc_store = InMemoryStore() # or use RedisStore for persistence

# Build the retriever
retriever = ParentDocumentRetriever(
    vectorstore=vectorstore,
    docstore=doc_store,
    child_splitter=child_splitter,
    parent_splitter=parent_splitter,
)

# Index the documents (creates both child and parent chunks)
retriever.add_documents(documents)

# At query time:
# 1. User question → embed → find top-k CHILD chunks
# 2. For each child chunk → look up its PARENT chunk
# 3. Deduplicate parents (multiple child chunks may share a parent)
# 4. Send PARENT chunks as context to LLM
results = retriever.get_relevant_documents("What is the refund policy?")
# results contain PARENT chunks, not the small child chunks
```

8.4 Pros, Cons & When to Use

Aspect	Details
■ Pros	Best combination of retrieval precision AND generation context. Handles long documents extremely well. Child chunks give sharp retrieval; parent chunks give the LLM full context.
■ Cons	More complex to set up. Requires two separate stores. Parent store needs persistence (InMemoryStore is lost on restart). Harder to debug.
Best used when	Long documents where questions require context beyond a single paragraph. Production systems where both precision and answer completeness are important. Technical manuals, long reports, books.
Avoid when	Simple documents where one chunk clearly contains the answer. Prototyping — adds complexity. When you need fast setup.
Recommended settings	child_size: 100-300 chars. parent_size: 800-2000 chars. child_overlap: 20-30 chars. Use RedisStore or SQLiteStore (not InMemoryStore) in production.

Chapter 9

Chunking Comparison

All strategies side by side — pick the right one for your use case

9.1 Full Feature Comparison

Strategy	Quality	Speed	Setup	AI Needed?	Best For
Fixed-Size (Character)	★★■	■	Trivial	No	Quick prototypes, uniform-text logs
Fixed-Size (Token)	★★★	■	Easy	No	Precise token control for LLM limits
Recursive Character	★★★	■	Easy	No	General-purpose default — most use cases
Sentence-Based (NLTK)	★★★	■	Easy	No	Prose documents — articles, reports
Sentence-Based (spaCy)	★★★★	■	Medium	No	Complex sentences, multilingual
Sentence Window	★★★★	■	Medium	No	Need precise retrieval + rich context
Semantic (SemanticChunker)	★★★★★	■	Medium	Yes (Embeddings)	Production — quality over speed
Markdown Header	★★★★	■	Easy	No	Docs, wikis, structured Markdown
HTML Header	★★★★	■	Easy	No	Web pages, HTML content
Proposition (Agentic)	★★★★★	■ (very slow)	Complex	Yes (LLM)	Legal, medical — max precision
Parent-Child	★★★★★	■	Complex	No	Long docs needing precision + context

9.2 Decision Flowchart

Your Situation	Recommended Strategy
Just learning RAG for the first time	RecursiveCharacterTextSplitter — simple, works well, zero dependencies

Your Situation	Recommended Strategy
Documents are Markdown (docs, wikis, READMEs)	MarkdownHeaderTextSplitter + RecursiveCharacterTextSplitter (two-stage)
Documents are HTML web pages	HTMLHeaderTextSplitter + RecursiveCharacterTextSplitter (two-stage)
Documents are standard PDFs with paragraphs	RecursiveCharacterTextSplitter with chunk_size=1000, overlap=200
Answer quality is critical, indexing speed is OK	SemanticChunker — best retrieval quality available
Long documents, need both precision and full context	Parent-Child (ParentDocumentRetriever)
Legal/medical/compliance — maximum accuracy required	Proposition Chunking with a capable LLM
Multilingual documents or complex sentence structure	SpacyTextSplitter
Need precise retrieval but rich generation context	Sentence Window Chunking

9.3 Chunk Size Recommendations by LLM

LLM	Context Window	Recommended Chunk Size	Max Chunks in Prompt
Ollama Phi-3 Mini	4,096 tokens	200-300 tokens per chunk	5-8 chunks
Ollama Mistral 7B	32,768 tokens	300-500 tokens per chunk	20-30 chunks
Ollama LLaMA 3.1 8B	131,072 tokens	400-600 tokens per chunk	50-100 chunks
OpenAI GPT-4o-mini	128,000 tokens	400-600 tokens per chunk	50-100 chunks
OpenAI GPT-4o	128,000 tokens	500-800 tokens per chunk	50-80 chunks
Claude Sonnet 3.7	200,000 tokens	500-1000 tokens per chunk	100-150 chunks
Claude Haiku 3.5	200,000 tokens	400-800 tokens per chunk	80-120 chunks

Chapter 10

What is a Vector Database?

Complete theory — from vectors to production-scale search

10.1 Starting From the Beginning — What is a Vector?

A vector is simply a list of numbers. In mathematics, a vector represents a point in multi-dimensional space. In AI and RAG, vectors represent the meaning of text.

Vectors

```
# A 2D vector (easy to visualise as a point on a graph):
point = [3.0, 4.0] # x=3, y=4

# A 384-dimensional vector (output of all-MiniLM-L6-v2):
text_vector = [0.23, -0.41, 0.87, 0.12, -0.05, 0.66, ...] # 384 numbers total

# Every chunk of text gets converted to one of these vectors
# The numbers encode the MEANING of the text, not its characters

# Similar texts → vectors that are mathematically close
vec_refund_1 = embed("How do I get a refund?")
vec_refund_2 = embed("What is the money-back policy?")
vec_weather = embed("What is the weather today?")

distance(vec_refund_1, vec_refund_2) = 0.12 # very close (similar meaning)
distance(vec_refund_1, vec_weather) = 0.87 # far apart (different topic)
```

10.2 What is a Vector Database?

A vector database is a specialised database built to store and search through millions of vectors efficiently. Unlike a regular database (which finds exact matches — WHERE name = 'John'), a vector database finds approximate nearest neighbours — the vectors most similar to a query vector.

Type	Query Style	Use Case	Example
Regular Database (SQL)	Exact match: WHERE city = 'London'	Store and retrieve structured records	MySQL, PostgreSQL, SQLite
Full-Text Search	Keyword match: MATCH 'refund policy'	Find documents containing exact words	Elasticsearch, Solr, Postgres FTS
Vector Database	Semantic match: find vectors near [0.23, -0.41, ...]	Find documents with similar MEANING	FAISS, Pinecone, Qdrant, Chroma

10.3 How Vector Search Works — ANN Algorithms

The naive approach to finding similar vectors is to compare your query vector against every stored vector one by one. This is called brute-force search and is $O(n)$ — it gets linearly slower as you add more vectors. For 1 million vectors, this is too slow.

Real vector databases use Approximate Nearest Neighbour (ANN) algorithms that trade a tiny amount of accuracy for massive speed gains:

Algorithm 1 — Flat (Brute Force)

Flat Index

```
# Flat/Brute-Force Search:
# Compare query against EVERY vector
#
# Pros: 100% exact results
# Cons:  $O(n)$  — 1M vectors = 1M comparisons
# Use: < 100,000 vectors where accuracy is critical
#
# FAISS index type: IndexFlatL2 (L2 distance) or IndexFlatIP (dot product)
import faiss
index = faiss.IndexFlatL2(384) # 384 = embedding dimensions
index.add(vectors)             # add all chunk vectors
D, I = index.search(query_vec, k=5) # find 5 nearest
```

Algorithm 2 — IVF (Inverted File Index)

IVF Index

```
# IVF Search:
# Step 1: Cluster all vectors into N groups (like filing cabinets)
# Step 2: At query time, only search the most relevant cabinets
#
# Pros: Much faster — only searches a fraction of vectors
# Cons: Approximate — might miss some true nearest neighbours
# Use: 100K to 10M vectors
#
# FAISS index type: IndexIVFFlat
import faiss
quantizer = faiss.IndexFlatL2(384)
index = faiss.IndexIVFFlat(quantizer, 384, 100) # 100 clusters
index.train(training_vectors) # must train IVF index first
index.add(vectors)
index.nprobe = 10 # search 10 of the 100 clusters at query time
D, I = index.search(query_vec, k=5)
```

Algorithm 3 — HNSW (Hierarchical Navigable Small World)

HNSW — Used by Most Production Vector DBs

```
# HNSW Search:
# Builds a graph where similar vectors are connected
# Navigation through the graph finds neighbours very fast
#
# Pros: Fastest query speed. No training needed. Best accuracy/speed tradeoff.
# Cons: High memory usage. Slower index building.
# Use: Production systems with millions of vectors
#
# Used by: Qdrant, Weaviate, Chroma (all use HNSW by default)
#
# Visual analogy: Like a city with highways (long connections at top level)
# and local streets (short connections at bottom level).
# You navigate from highway to street to reach your exact destination fast.
```

Algorithm 4 — Product Quantisation (PQ)

Product Quantisation

```
# Product Quantisation:
# Compresses vectors to use much less memory
# A 384-dim float32 vector = 384 * 4 bytes = 1,536 bytes
# With PQ compression = 64-256 bytes (10-20x smaller)
#
# Trade-off: Smaller memory, slightly less accurate search
# Use: When you have billions of vectors and RAM is limited
#
# FAISS index type: IndexIVFPQ
index = faiss.IndexIVFPQ(quantizer, 384, 100, 16, 8)
# 100 clusters, 16 sub-quantisers, 8 bits each
```

10.4 Key Vector Database Concepts

Concept	What It Means	Why It Matters
Embedding Dimensions	The number of numbers in each vector (e.g., 384 for all-MiniLM, 1536 for text-embedding-3-small)	Higher dimensions = more expressive but more memory. All vectors in one index must have the same dimensions.
Distance Metric	How 'closeness' between two vectors is measured	L2 (Euclidean): straight-line distance. Cosine: angle between vectors (ignores magnitude). Dot product: similar to cosine for normalised vectors. Cosine is most common for text.
k (top-k)	How many similar vectors to return for each query	k=3 returns 3 chunks. Too small = miss relevant info. Too large = include irrelevant noise. k=4-6 is typical.
Namespace/Collection	A logical partition within a vector database	Like a table in SQL. Store vectors for different documents or users in separate namespaces to avoid cross-contamination.
Metadata filtering	Add structured data alongside each vector and filter by it at query time	Store {source: 'contract.pdf', page: 5, date: '2024'} with each vector. Then query: find vectors similar to X WHERE source='contract.pdf'.
Upsert	Insert if new, update if ID already exists	Allows you to update one chunk without rebuilding the entire index. Essential for keeping your knowledge base current.

Chapter 11

7 Vector Databases Compared

FAISS · Chroma · Qdrant · Pinecone · Weaviate · pgvector · Redis

FAISS — The fastest in-memory vector search library — perfect for starting out

Property	Value
Made by	Meta AI Research
Type	Library (not a database server)
Persistence	Manual (save_local / load_local)
Scale	Up to ~10M vectors comfortably
Setup command	pip install faiss-cpu
Hosting	In your Python process
Metadata filtering	Basic (post-filter on metadata)
Cost	Free (open source)

Strengths	Weaknesses
✓ Zero infrastructure — runs inside your Python app	✗ In-memory only by default — data lost on restart without save_local()
✓ Fastest raw search speed for small-medium datasets	✗ No built-in server — cannot be shared between multiple processes/machines
✓ Multiple index types (Flat, IVF, HNSW, PQ)	✗ Limited metadata filtering (you filter after retrieval, not during)
✓ Battle-tested by Meta AI at massive scale	✗ No built-in replication or high availability
✓ Native LangChain integration — easiest setup	

■ **Use when:** Prototyping, learning RAG, single-machine apps, when you have < 1M vectors and want zero infrastructure

FAISS — LangChain Code

```
from langchain_community.vectorstores import FAISS

# Build from documents
vectorstore = FAISS.from_documents(chunks, embeddings)

# Save to disk (important!)
```

```
vectorstore.save_local("./faiss_index")

# Load from disk on next startup
vectorstore = FAISS.load_local(
    "./faiss_index", embeddings,
    allow_dangerous_deserialization=True
)

# Search
docs = vectorstore.similarity_search("refund policy", k=4)
docs = vectorstore.max_marginal_relevance_search("refund policy", k=4)
```

Chroma — The easiest persistent vector database — zero infrastructure, works locally

Property	Value
Made by	Chroma (open source)
Type	Embedded database (like SQLite for vectors)
Persistence	Automatic to local disk
Scale	Up to ~1M vectors reliably
Setup command	pip install chromadb
Hosting	Local disk (or Chroma Cloud)
Metadata filtering	Good — metadata filters at query time
Cost	Free (open source) / Chroma Cloud (paid)

Strengths	Weaknesses
✓ Persists automatically to disk — no save/load calls needed	✗ Slower than FAISS for pure search speed
✓ Easiest setup of any persistent vector store	✗ Less production-tested than Qdrant or Pinecone
✓ Good metadata filtering support	✗ Limited scaling beyond ~1M vectors
✓ Can run as embedded DB or as a separate server	
✓ Built-in multi-tenancy (collections)	

■ **Use when:** Small production apps that need persistence without running a separate database server. The natural step up from FAISS.

Chroma — LangChain Code

```
from langchain_community.vectorstores import Chroma

# Build — automatically persists to disk
vectorstore = Chroma.from_documents(
    chunks,
    embeddings,
    persist_directory="./chroma_db",  # saves here automatically
    collection_name="my_documents"
```

```
)

# Load existing (same code — detects existing DB automatically)
vectorstore = Chroma(
    persist_directory="./chroma_db",
    embedding_function=embeddings,
    collection_name="my_documents"
)

# Search with metadata filter
docs = vectorstore.similarity_search(
    "refund policy",
    k=4,
    filter={"source": "contract.pdf"} # filter by metadata
)
```

Qdrant — Production-grade vector database with excellent filtering — the best self-hosted option

Property	Value
Made by	Qdrant (open source + cloud)
Type	Standalone database server
Persistence	Yes — built-in, reliable
Scale	Billions of vectors
Setup command	Docker: docker run -p 6333:6333 qdrant/qdrant
Hosting	Self-hosted (Docker) or Qdrant Cloud
Metadata filtering	Excellent — rich filter conditions at query time
Cost	Free (self-hosted) / Qdrant Cloud (paid)

Strengths	Weaknesses
✓ HNSW index by default — very fast search	✗ Requires running a separate Docker container
✓ Rich metadata filtering (AND/OR/NOT, range filters, nested fields)	✗ More complex setup than FAISS or Chroma
✓ Built-in payload (metadata) storage alongside vectors	✗ Qdrant Cloud can get expensive at high scale
✓ Excellent documentation and active development	
✓ Supports named vectors (multiple embeddings per point)	
✓ Handles billions of vectors with proper sharding	

■ **Use when:** *Production applications that outgrow Chroma. When you need rich metadata filtering. When you need self-hosted control with production reliability.*

Qdrant — LangChain Code

```
# Start Qdrant: docker run -p 6333:6333 qdrant/qdrant
# pip install qdrant-client

from langchain_community.vectorstores import Qdrant
from qdrant_client import QdrantClient

client = QdrantClient(host="localhost", port=6333)

vectorstore = Qdrant.from_documents(
    chunks,
    embeddings,
    url="http://localhost:6333",
    collection_name="my_docs",
    force_recreate=False # set True to wipe and rebuild
)

# Rich filter at query time:
from qdrant_client.models import Filter, FieldCondition, MatchValue

docs = vectorstore.similarity_search(
    "refund policy",
    k=4,
    filter=Filter(must=[
        FieldCondition(key="metadata.source", match=MatchValue(value="contract.pdf")),
        FieldCondition(key="metadata.page", range={"gte": 1, "lte": 10})
    ])
)
```

Pinecone — Fully managed cloud vector database — zero ops, unlimited scale

Property	Value
Made by	Pinecone Systems (commercial)
Type	Fully managed cloud service
Persistence	Yes — fully managed
Scale	Unlimited (managed by Pinecone)
Setup command	API key only — no server to run
Hosting	Pinecone cloud (AWS/GCP/Azure regions)
Metadata filtering	Good — metadata filtering
Cost	Free tier (100K vectors) / ~\$70/month for production

Strengths	Weaknesses
✓ Zero infrastructure — no servers to manage	✗ Cost — gets expensive at high vector counts
✓ Auto-scaling — handles traffic spikes automatically	✗ Data leaves your infrastructure — privacy concern
✓ High availability and redundancy built in	✗ Vendor lock-in
✓ 99.99% SLA for production indexes	✗ Less flexible than self-hosted alternatives

✓ Real-time updates — add/delete vectors instantly	
✓ Simple API — easy to get started	

■ **Use when:** Enterprise production apps where you want zero infrastructure management. When your team does not want to run and maintain a database. When uptime guarantees are required.

Pinecone — LangChain Code

```
# pip install pinecone-client langchain-pinecone
# Get API key from: https://app.pinecone.io

from langchain_pinecone import PineconeVectorStore
import pinecone

pc = pinecone.Pinecone(api_key="your-api-key")

# Create index (do this once)
pc.create_index(
    name="my-rag-index",
    dimension=384,          # must match your embedding model dimensions
    metric="cosine",
    spec=pinecone.ServerlessSpec(cloud="aws", region="us-east-1")
)

# Build from documents
vectorstore = PineconeVectorStore.from_documents(
    chunks,
    embeddings,
    index_name="my-rag-index"
)

# Load existing index
vectorstore = PineconeVectorStore.from_existing_index(
    index_name="my-rag-index",
    embedding=embeddings
)

docs = vectorstore.similarity_search("refund policy", k=4)
```

pgvector — Vector search inside your existing PostgreSQL — zero new infrastructure

Property	Value
Made by	PostgreSQL extension (open source)
Type	PostgreSQL extension
Persistence	Yes — PostgreSQL handles it
Scale	~10M vectors efficiently
Setup command	CREATE EXTENSION vector; (in PostgreSQL)
Hosting	Wherever your PostgreSQL runs

Property	Value
Metadata filtering	Full SQL — any query you can write
Cost	Free (open source)

Strengths	Weaknesses
✓ No new infrastructure if you already use PostgreSQL	✗ Slower than dedicated vector databases at large scale (> 5M vectors)
✓ Full SQL queries on metadata — use JOINS, WHERE, ORDER BY	✗ Requires PostgreSQL setup if you don't already have it
✓ ACID transactions — vector operations are atomic	✗ HNSW index only added in recent versions — older versions use brute force
✓ Combine vector search with SQL filtering in one query	
✓ Use existing PostgreSQL backups, replication, security	

■ **Use when:** Your app already uses PostgreSQL. You want SQL and vector search in one place. You want ACID compliance for vector operations.

pgvector — LangChain Code

```
# pip install psycpg2-binary pgvector langchain-community
# PostgreSQL must have pgvector extension:
# CREATE EXTENSION IF NOT EXISTS vector;

from langchain_community.vectorstores import PGVector

CONNECTION_STRING = "postgresql://user:password@localhost:5432/mydb"

# Build from documents
vectorstore = PGVector.from_documents(
    chunks,
    embeddings,
    connection_string=CONNECTION_STRING,
    collection_name="my_embeddings"
)

# The power of pgvector: combine with SQL
# All metadata is queryable with full SQL
docs = vectorstore.similarity_search(
    "refund policy",
    k=4,
    filter={"source": "contract.pdf"}
)

# Under the hood, this runs SQL like:
# SELECT *, 1 - (embedding <=> query_vec) AS similarity
# FROM embeddings
# WHERE metadata->>'source' = 'contract.pdf'
# ORDER BY similarity DESC LIMIT 4
```

Weaviate — GraphQL-powered vector database with hybrid search built in

Property	Value
Made by	Weaviate (open source + cloud)
Type	Standalone database server
Persistence	Yes — built-in
Scale	Billions of vectors
Setup command	Docker or Weaviate Cloud
Hosting	Self-hosted or Weaviate Cloud
Metadata filtering	Excellent — GraphQL + BM25 hybrid built-in
Cost	Free (self-hosted) / Weaviate Cloud (paid)

Strengths	Weaknesses
✓ Built-in hybrid search (BM25 + vector) out of the box	✗ Steeper learning curve — GraphQL schema setup
✓ GraphQL API — very powerful querying	✗ Heavier setup than Qdrant
✓ Schema-based — enforces data types on metadata	✗ GraphQL may be unfamiliar to many developers
✓ Multi-modal — supports text, images, video vectors	
✓ Auto-vectorisation — can embed data automatically	

■ **Use when:** When you want hybrid search (dense + keyword) built in. When you need GraphQL APIs. Multi-modal applications.

Weaviate — LangChain Code

```
# pip install weaviate-client langchain-weaviate
# Start: docker run -p 8080:8080 semitechnologies/weaviate

import weaviate
from langchain_weaviate import WeaviateVectorStore

client = weaviate.connect_to_local()

vectorstore = WeaviateVectorStore.from_documents(
    chunks,
    embeddings,
    client=client,
    index_name="MyDocuments"
)

# Weaviate has built-in hybrid search (BM25 + vector):
docs = vectorstore.similarity_search(
    "refund policy",
    k=4,
    search_type="hybrid", # combines vector + keyword search
    alpha=0.5             # 0=pure keyword, 1=pure vector, 0.5=balanced
)
```

Redis Vector — Ultra-low latency vector search — best when speed is everything

Property	Value
Made by	Redis (open source + cloud)
Type	In-memory database with persistence
Persistence	Yes — Redis persistence (RDB/AOF)
Scale	~100M vectors (all in RAM)
Setup command	Docker: <code>docker run -p 6379:6379 redis/redis-stack</code>
Hosting	Self-hosted or Redis Cloud
Metadata filtering	Good — tag and numeric filters
Cost	Free (self-hosted) / Redis Cloud (paid)

Strengths	Weaknesses
✓ Lowest latency of any vector database — everything in RAM	✗ Everything in RAM — expensive at large scale
✓ Sub-millisecond search times	✗ Less mature vector feature set than Qdrant/Weaviate
✓ Familiar Redis API if your team already uses Redis	✗ RAM limits your vector count
✓ Combined caching + vector search in one service	

■ **Use when:** When query latency is the top priority. When you already use Redis for caching. Real-time applications needing sub-millisecond responses.

Redis Vector — LangChain Code

```
# pip install redis langchain-community
# docker run -p 6379:6379 redis/redis-stack

from langchain_community.vectorstores import Redis

vectorstore = Redis.from_documents(
    chunks,
    embeddings,
    redis_url="redis://localhost:6379",
    index_name="my_docs"
)

docs = vectorstore.similarity_search("refund policy", k=4)
```

Chapter 12

Choosing the Right Combination

Recommended stacks for every use case

12.1 Full Comparison — All 7 Vector Databases

Database	Persistence	Scale	Setup Effort	Cost	Filtering	Best For
FAISS	Manual (save/load)	~10M	Zero	Free	Limited	Learning, prototypes
Chroma	Automatic	~1M	Minimal	Free	Good	Small production apps
Qdrant	Built-in	Billions	Medium	Free+Cloud	Excellent	Mid-large production
Pinecone	Managed	Unlimited	Low (API key)	\$70/mo+	Good	Enterprise, zero ops
pgvector	PostgreSQL	~10M	Low (if Postgres exists)	Free	Full SQL	Postgres-native apps
Weaviate	Built-in	Billions	Medium	Free+Cloud	GraphQL	Hybrid search, multi-modal
Redis	In-memory+persist	~100M	Low	Free+Cloud	Good	Ultra-low latency

12.2 Recommended Stacks by Use Case

Use Case	Chunking Strategy	Embedding	Vector DB	LLM
Learning RAG for first time	RecursiveCharacterTextSplitter (500 chars, 50 overlap)	all-MiniLM-L6-v2 (free, local)	FAISS (zero setup)	Ollama + Phi-3 or Gemma3:4b
Small production app (< 100K docs)	RecursiveCharacterTextSplitter (1000 chars, 200 overlap)	BAAI/bge-large-en-v1.5 (free, best local)	Chroma (auto-persist to disk)	GPT-4o-mini or Claude Haiku 3.5
Documentation / wiki knowledge base	MarkdownHeaderTextSplitter + RecursiveCharacterTextSplitter	BAAI/bge-large-en-v1.5	Qdrant (Docker)	GPT-4o-mini or Claude Sonnet
Legal / medical — max accuracy	Proposition Chunking + Semantic Chunking	voyage-3 or text-embedding-3-large	Qdrant or Pinecone	Claude Sonnet 3.7 (200k context)
Long documents (50+ pages each)	Parent-Child + SemanticChunker for children	text-embedding-3-small	Qdrant or Chroma	Claude Sonnet 3.7
Air-gapped / data cannot leave network	RecursiveCharacterTextSplitter (tokens)	BAAI/bge-large-en-v1.5 (local)	FAISS (save to disk) or Qdrant (on-prem)	Ollama + Mistral 7B or LLaMA 3.1 8B

Use Case	Chunking Strategy	Embedding	Vector DB	LLM
Already using PostgreSQL	RecursiveCharacterTextSplitter	text-embedding-3-small or HuggingFace	pgvector (zero new infrastructure)	GPT-4o-mini
Enterprise — zero infrastructure management	SemanticChunker	text-embedding-3-large	Pinecone (fully managed)	GPT-4o or Claude Sonnet 3.7

12.3 Quick Decision Tree

Question	If YES → use this	If NO → next question
Just learning RAG?	FAISS + RecursiveCharacter + all-MiniLM-L6-v2 + Ollama	Continue...
Documents have Markdown/HTML headers?	MarkdownHeaderTextSplitter or HTMLHeaderTextSplitter	Continue...
Answer quality is top priority (slow indexing OK)?	SemanticChunker + BAAI/bge-large-en-v1.5	Continue...
Long docs where context > single paragraph matters?	Parent-Child Retriever	Continue...
Already running PostgreSQL?	pgvector — no new infra	Continue...
Need zero infrastructure (fully managed)?	Pinecone	Continue...
Need self-hosted + production scale?	Qdrant (Docker)	Continue...
Small production app, need auto-persistence?	Chroma	Use FAISS with save_local()